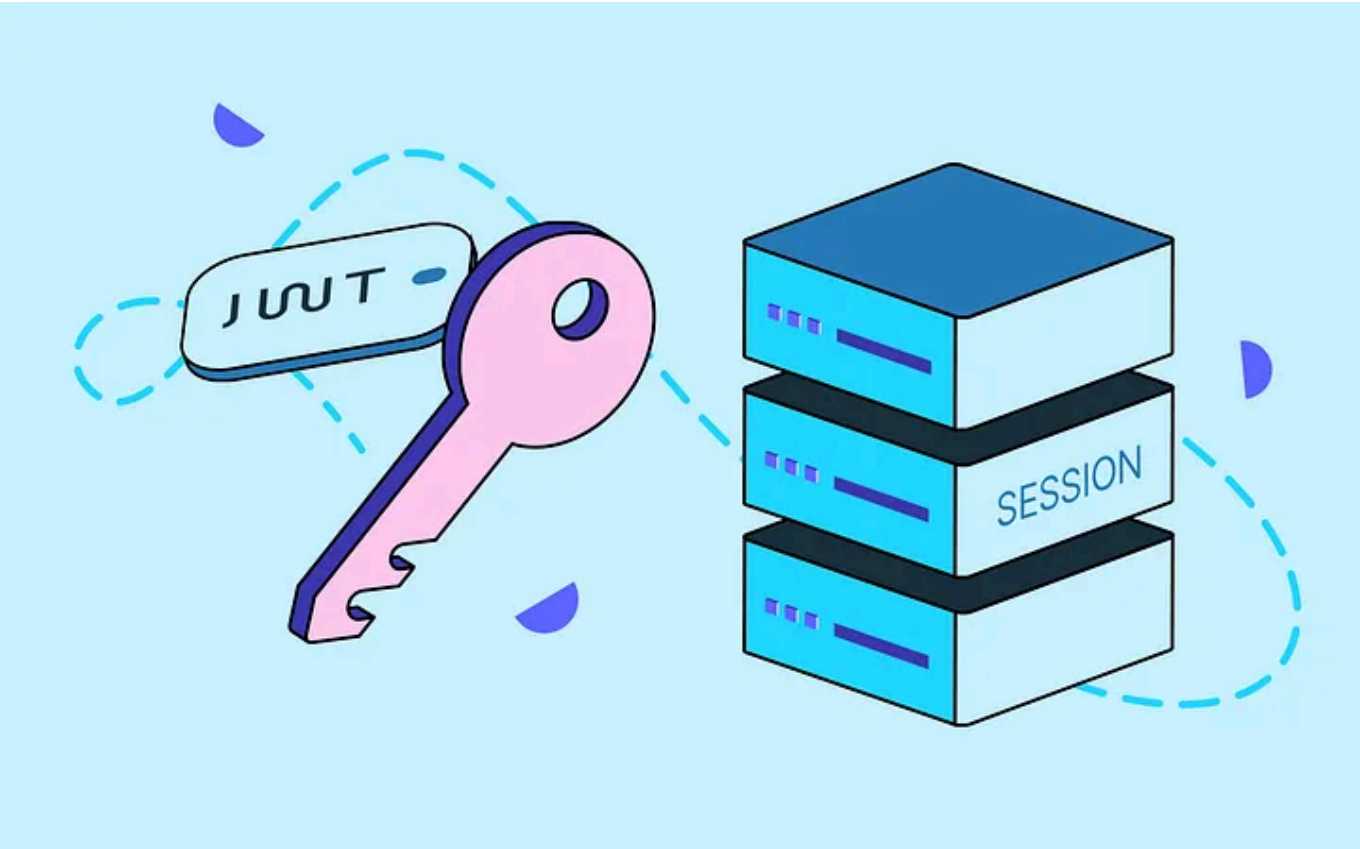
Express Session Management

(Node Series Part 11)

Shubham Gupta Follow 12 min read · Aug 9, 2025

6 1



Session management is the process of securely tracking and maintaining a user's interactions with a web application across multiple HTTP requests.

Since HTTP is stateless (each request is independent), sessions provide a way to preserve user-specific data (like login status, preferences, or shopping cart items) during their visit.

Here's why it's used:

Maintain User State Across Requests: HTTP is stateless, meaning each request is independent. Sessions help track user activity (e.g., login status, preferences) across multiple pages.

Authentication & Authorization: Sessions store user credentials after login, allowing the server to verify identity and permissions for subsequent requests without re-authenticating.

Personalization: Sessions remember user-specific data (e.g., shopping carts, themes, language preferences) for a seamless experience.

Security

- Session IDs replace sensitive data (like passwords) with temporary tokens.
- Timeouts automatically expire sessions to reduce hijacking risks.
- Secure Cookies/HTTPS prevent session theft via encryption.

Performance Optimization: Caching session data reduces repeated database queries, improving speed.

. . .

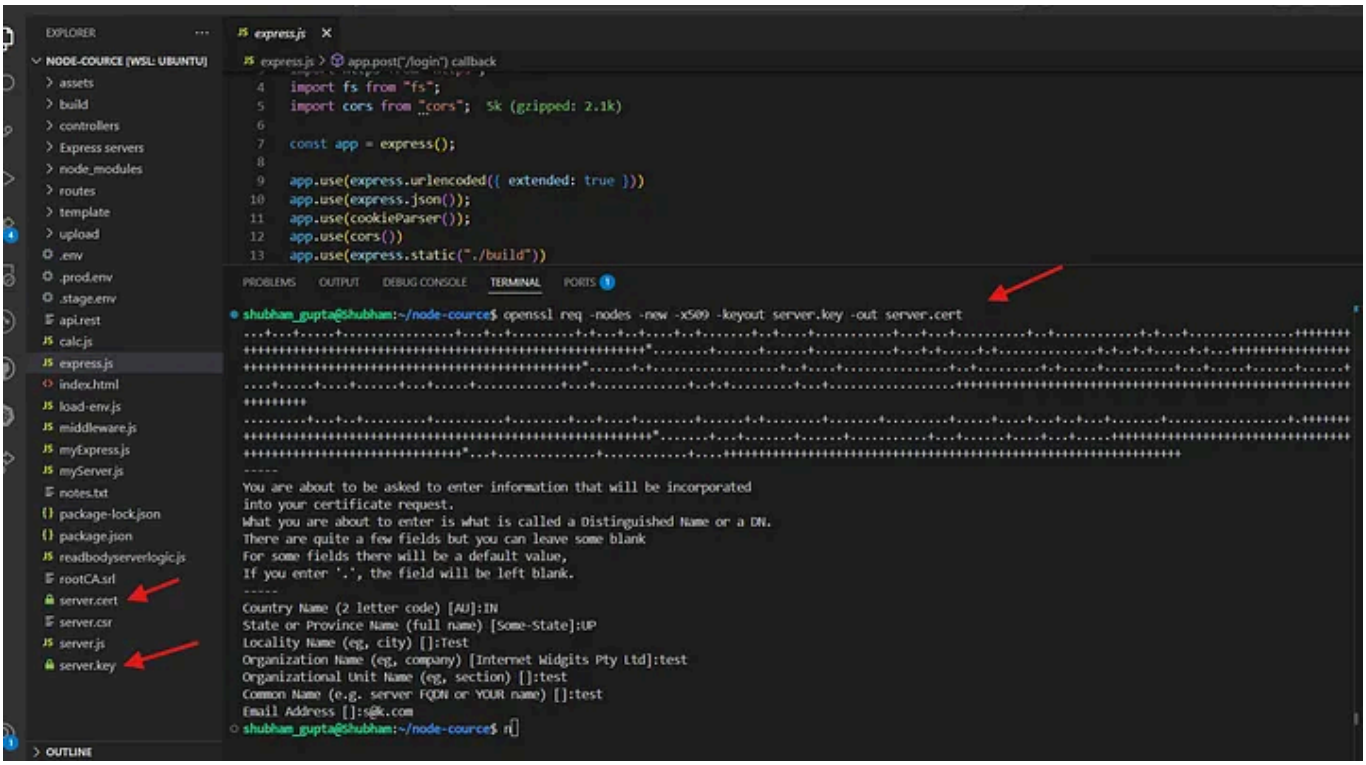
Implementing session

Here, I have created a server and a React app to demonstrate a user session.

For this, u need to set up an HTTPS server we will need an SSL/TLS certificate.

First, create a self-signed certificate using OpenSSL, run the following command in the terminal

```
openssl req -nodes -new -x509 -keyout server.key -out server.cert
```



After that, you will get the server key and a server certificate, which will be used to create our HTTPS server

Now we create a new server in Express.js to create an HTTPS server

```
// HTTPS Server Setup
const options = {
    key: fs.readFileSync("server.key"),     // Private key file
    cert: fs.readFileSync("server.cert")    // SSL certificate file
};

// Create HTTPS server
const PORT = 3000;
https.createServer(options, app).listen(PORT, () => {
    console.log(`🔒 HTTPS Server running on https://localhost:${PORT}`);
});
```

Here is the full server setup with login and dashboard api

```
import express from "express";
import cookieParser from "cookie-parser";
import https from "https";
import fs from "fs";
import cors from "cors";
const user = {
    username: "test",
    password: "123"
}
const app = express();

app.use(express.urlencoded({ extended: true }))
app.use(express.json());
app.use(cookieParser());
app.use(cors({
  origin: ['http://localhost:3001'], //add your React dev server
  credentials: true
}));

app.post("/api/login", (req, res) => {
    const { username, password } = req.body
    if (username === user.username && user.password === password) {
        res.send({ success: true });
    } else {
        res.send({ success: false,message:"Invalid User" });
    }

});
```

```
app.get("/api/dashboard", (req, res) => {
    res.send({ success: true,message:"Dashboard Data User is valid" });
});

// HTTPS Server Setup
const options = {
    key: fs.readFileSync("server.key"),    // Private key
    cert: fs.readFileSync("server.cert"), // Certificate
};

//to create https server
const PORT = 5000;
https.createServer(options, app).listen(PORT, () => {
    console.log(`HTTPS Server running on https://localhost:${PORT}`);
});
```

Now I created a simple React app and generated a build, and then copied the build to my Node project

```
import axios from "axios";
import { useState, useEffect } from "react";
import { BrowserRouter, Routes, Route,
 useNavigate, useLocation, Navigate } from "react-router-dom";

// Configure axios
axios.defaults.baseURL = "https://localhost:5000";
axios.defaults.withCredentials = true;

function TestApp() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("");
  const [dashboardData, setDashboardData] = useState(null);
  const navigate = useNavigate();
  const location = useLocation();

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      const response = await axios.post("/api/login",
{ username, password });
      setMessage(response.data.message);
```

```
        if (response.data.success) {
          navigate("/dashboard");
        }
      } catch (err) {
        setMessage(err.response?.data?.message || "Login failed");
      }
    };

    const getData = async () => {
      try {
        const response = await axios.get("/api/dashboard");
        if (response.data.success) {
          setDashboardData(response.data.message);
        } else {
          setMessage(response.data.message
  || "Session expired - please login");
        }
      } catch (err) {
        setMessage(err.response?.data?.message
 || "Session expired - please login");
        navigate("/login");
      }
    };

    useEffect(() => {
      if (location.pathname === "/dashboard") {
        getData();
      }
    }, [location]);

    return (
      <div className="app">
        {message && <div className="message">{message}</div>}

        <Routes>
          <Route
            path="/login"
            element={
              <div className="login-form">
                <h2>Login</h2>
                <form onSubmit={handleLogin}>
                  <input
                    type="text"
                    placeholder="Username"
                    value={username}
                    onChange={(e) => setUsername(e.target.value)}
                    required
                  />
                  <input
                    type="password"
```

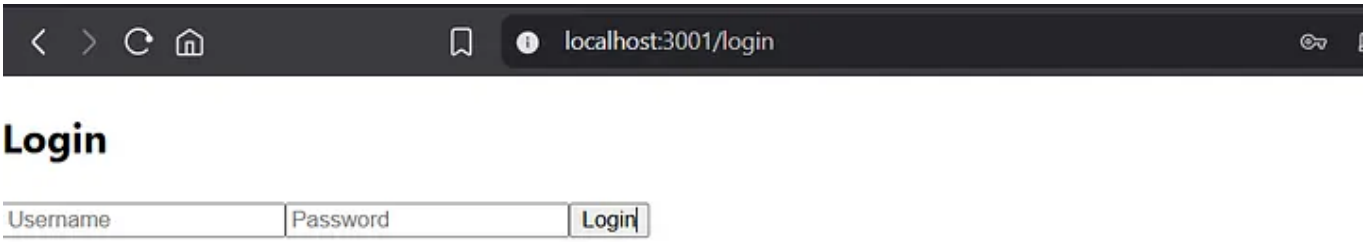
```
                  placeholder="Password"
                  value={password}
                  onChange={(e) => setPassword(e.target.value)}
                  required
                />
                <button type="submit">Login</button>
              </form>
            </div>
          }
        />

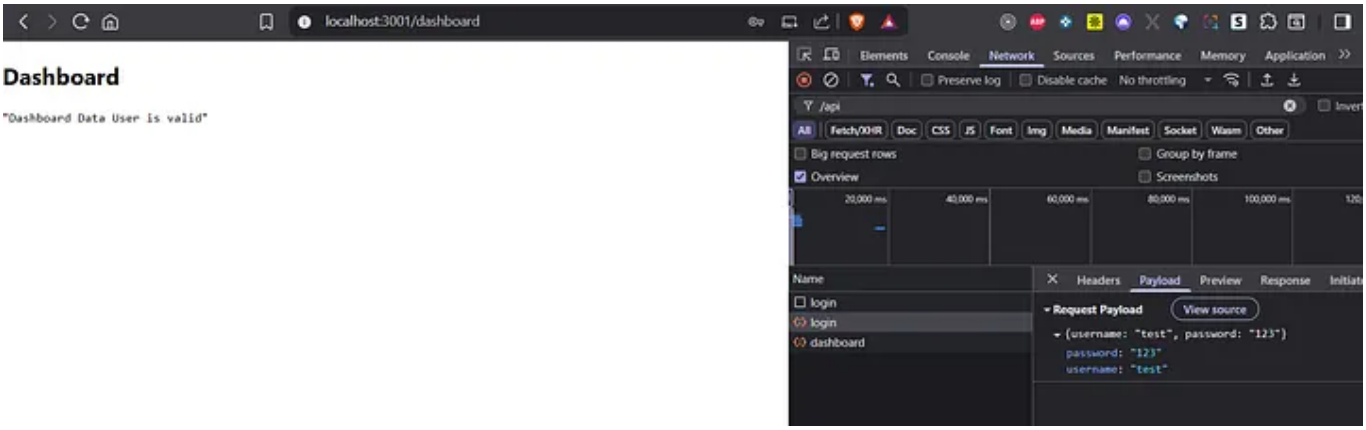
        <Route
          path="/dashboard"
          element={
            <div className="dashboard">
              <h2>Dashboard</h2>
              {dashboardData}
            </div>
          }
        />
        <Route path="*" element={<Navigate to="/login" replace />} />
      </Routes>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <TestApp />
    </BrowserRouter>
  );
}

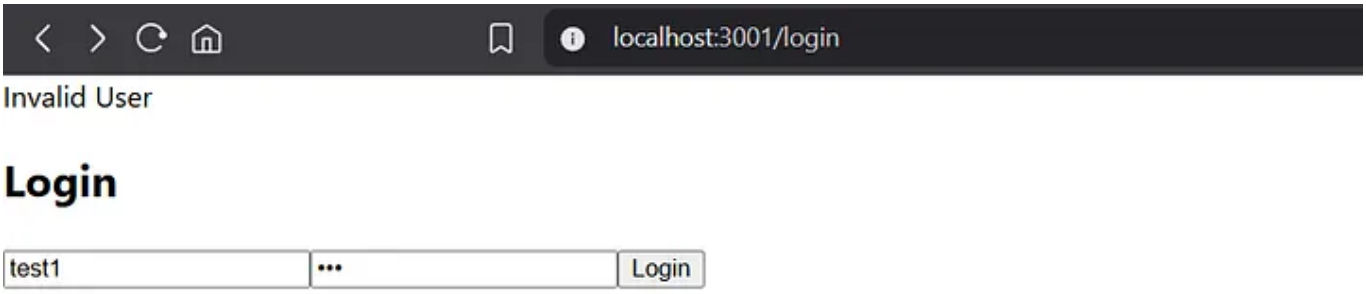
export default App;
```

```
npm start // start your react app
```





Check for an invalid user



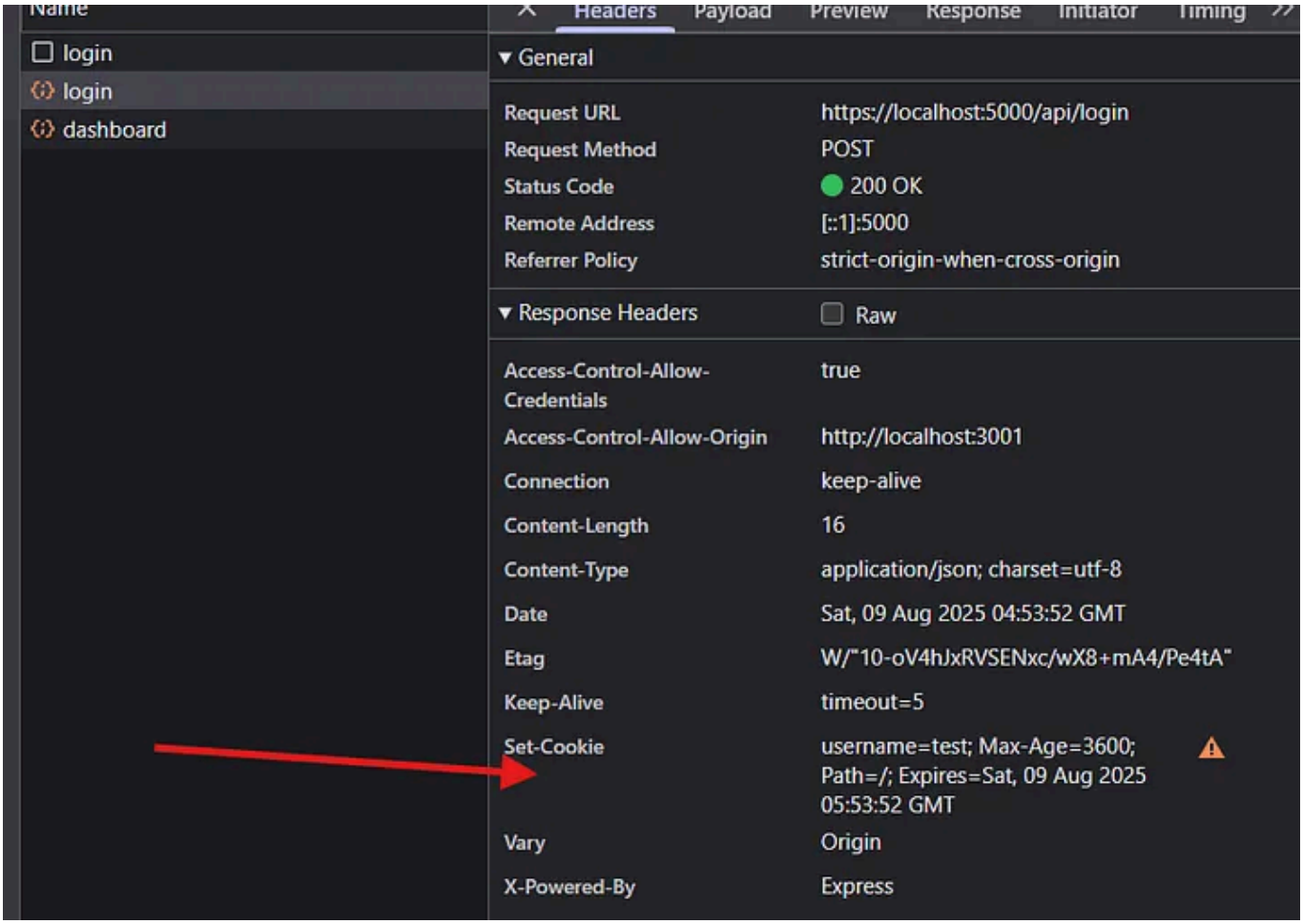
Working as expected. Our basic setup is done. Now let’s add some cookies to maintain the session.

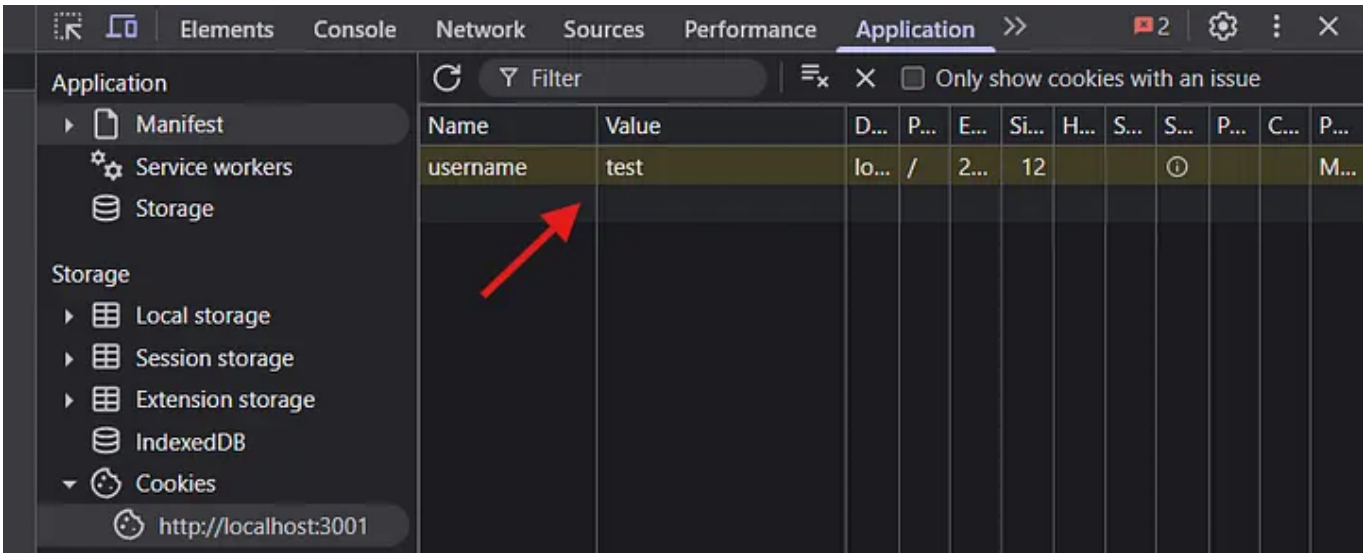
Adding a cookie in response when logging in

```
app.post("/api/login", (req, res) => {
    const { username, password } = req.body
    if (username === user.username && user.password === password) {
         res.cookie("username", username, {
            httpOnly: true,
            secure: true,       // Must be true for HTTPS
            sameSite: 'none', maxAge: 1000 * 60 * 60
        })
        res.send({ success: true });
    } else {
        res.send({ success: false, message: "Invalid User" });
    }

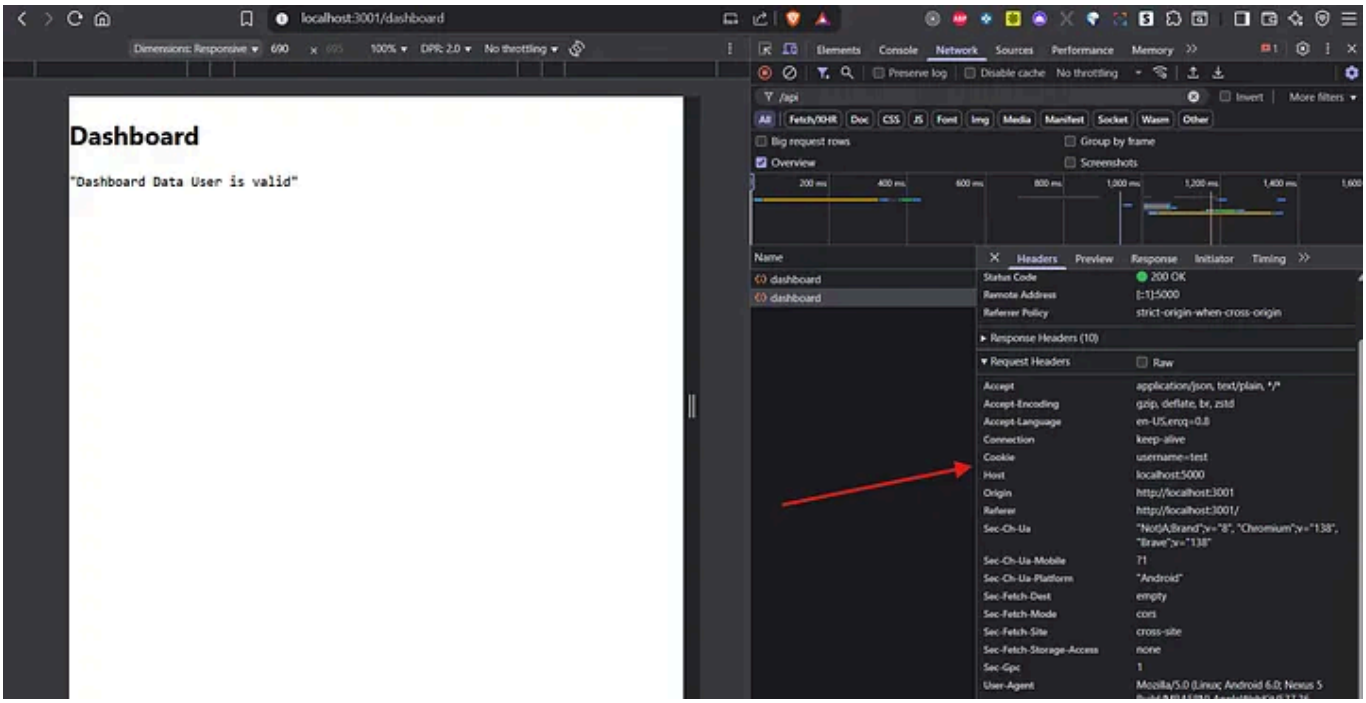
});
```

Now when u log in agai,n you can see the cookie in response and see cookies in storage





And when /dashboard api is called cookie is automatically passed

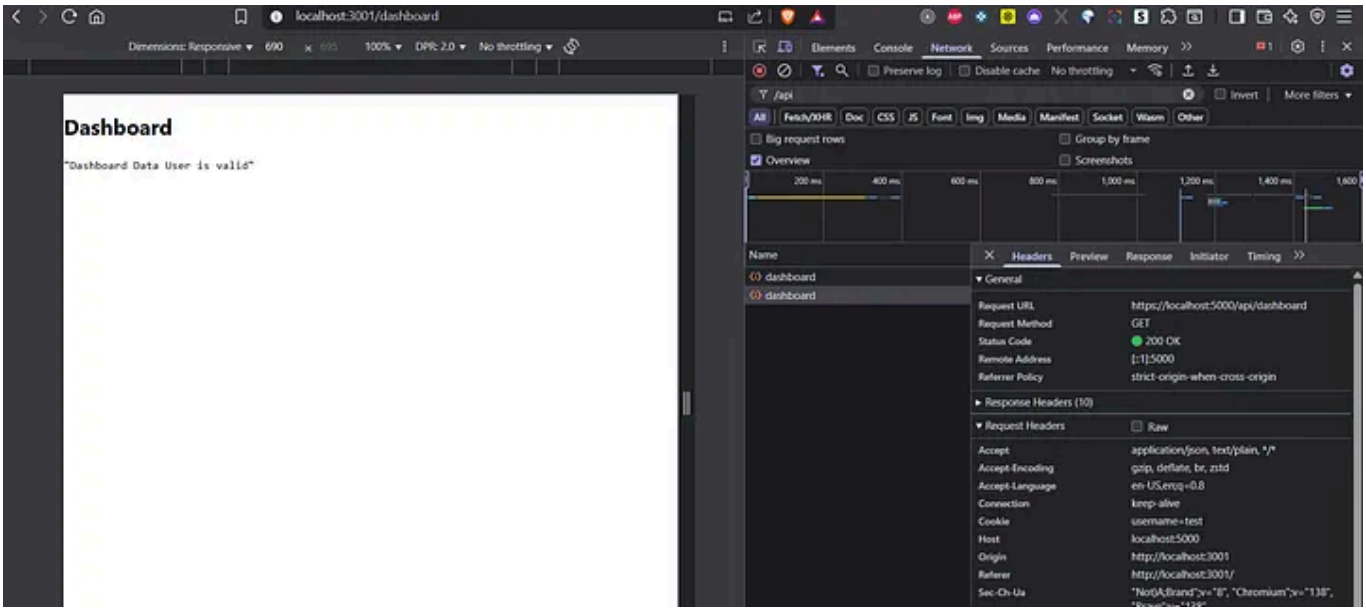


Now let's modify the /dashboard , so that when cookie is present we will show data else erorr

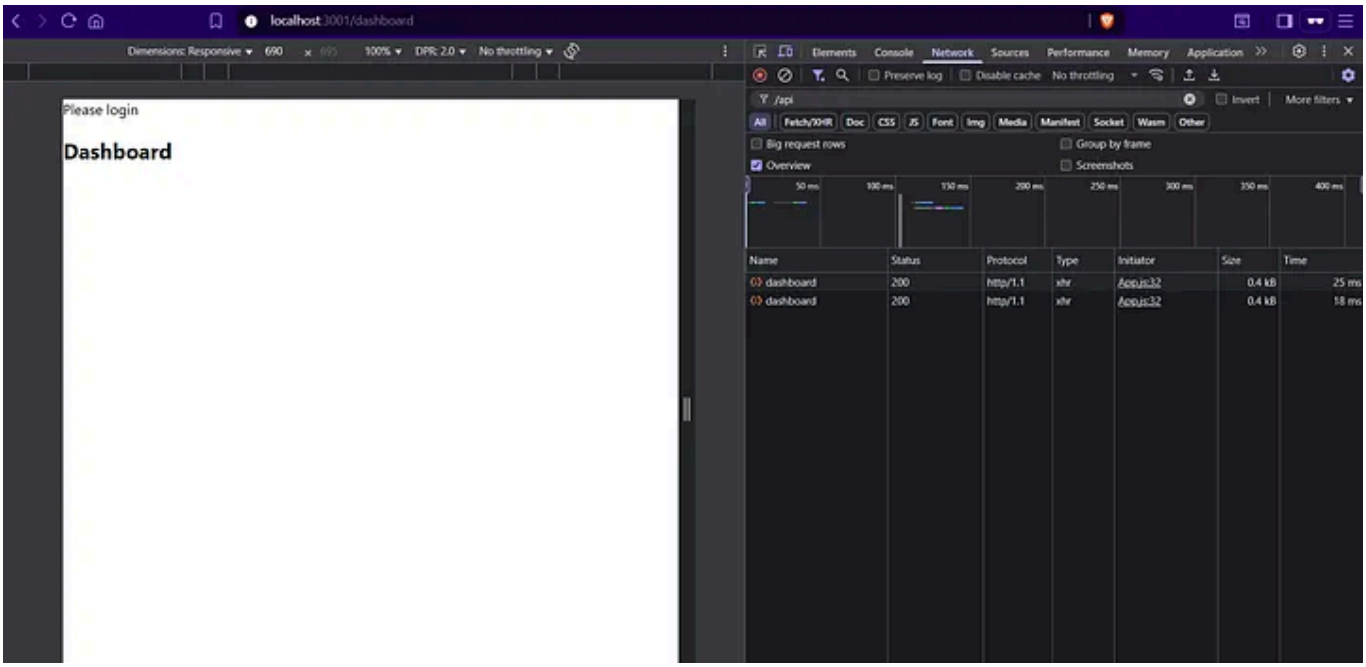
```
app.get("/api/dashboard", (req, res) => {
    const cookie = req.cookies || {}
```

```
    const { username } = cookie
    if (username) {
      res.send({ success: true, message: "Dashboard Data User is valid" });

    } else {
      res.send({ success: false, message: "Please login " });
    }
});
```



Now, copy the URL and open it in incognito



Now in the above code we maintained a session and here we have to do it manually, so we will use a express library.

. . .

Express-session

What does it do?

Session management helps track user interactions across requests. The express-session middleware stores session data on the server.

```
npm install express-session
```

Add config as middleware.

Express-session uses cookies to save and manage sessions. It manages all the functionalities for us.

For each unique session, we get a unique ID and signatur,e which is made using a secret key. so we need to pass a random and long secret key.

```
connect.sid=s:XkS92ixd8Fz...9d.7eK8O3NL4tcYzt6XGvQ1cgUZzIE

s:<session-id>.<signature>
```

Express uses the ***cookie-signature*** package internally to generate a session ID.

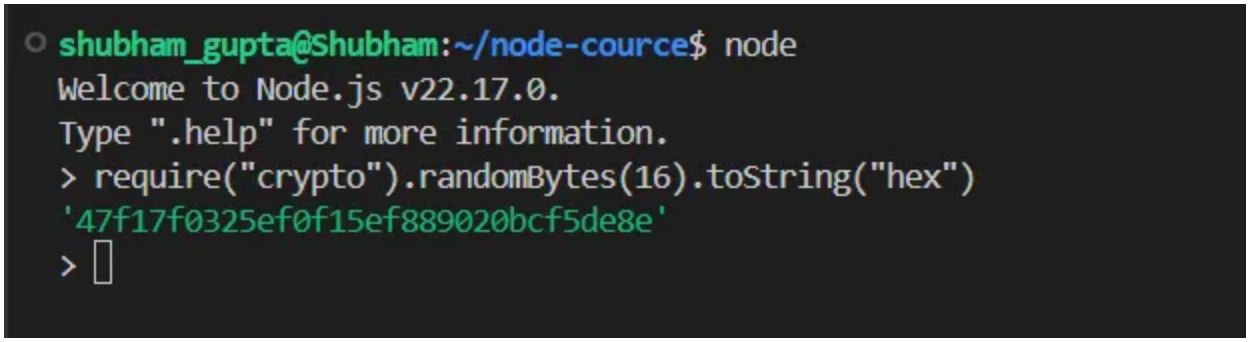
```
const crypto = require('crypto');
const signature = crypto
  .createHmac('sha256', secret)
  .update(sessionID)
  .digest('base64')
```

This unique string is then passed to the client and checked at our end.

Let's use this inside our express.js file

You need a random string to pass into the function, so open node cli and run the below command to get a random string and copy it.

```
require("crypto").randomBytes(16).toString("hex")
```




```
import express from "express";
import cookieParser from "cookie-parser";
import https from "https";
import fs from "fs";
import cors from "cors";
import session from "express-session"
const user = {
    username: "test",
    password: "123"
}
const app = express();

app.use(session(
    {
        secret: "47f17f0325ef0f15ef889020bcf5de8e",//random key
        resave: false,
        saveUninitialized:false,
        cookie: {
            maxAge: 1000 * 60 * 60,
            httpOnly: true,
            secure: true,       // Must be true for HTTPS
            sameSite: 'none',
        }
    }

));
app.use(express.urlencoded({ extended: true }))
app.use(express.json());
app.use(cookieParser());
app.use(cors({
```

```
    origin: 'http://localhost:3001', //add your React dev server
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization']
}));

app.post("/api/login", (req, res) => {
    const { username, password } = req.body
    if (username === user.username && user.password === password) {
        // will tie up unique seesion id here i use username,and id
        // so that it can be used on new api call to indetify the user
        req.session.user = { username,id:123 }
        res.send({ success: true });
    } else {
        res.send({ success: false, message: "Invalid User" });
    }

});

app.get("/api/dashboard", (req, res) => {
    const cookie = req.cookies || {}
    const { username } = cookie
    console.log(cookie);

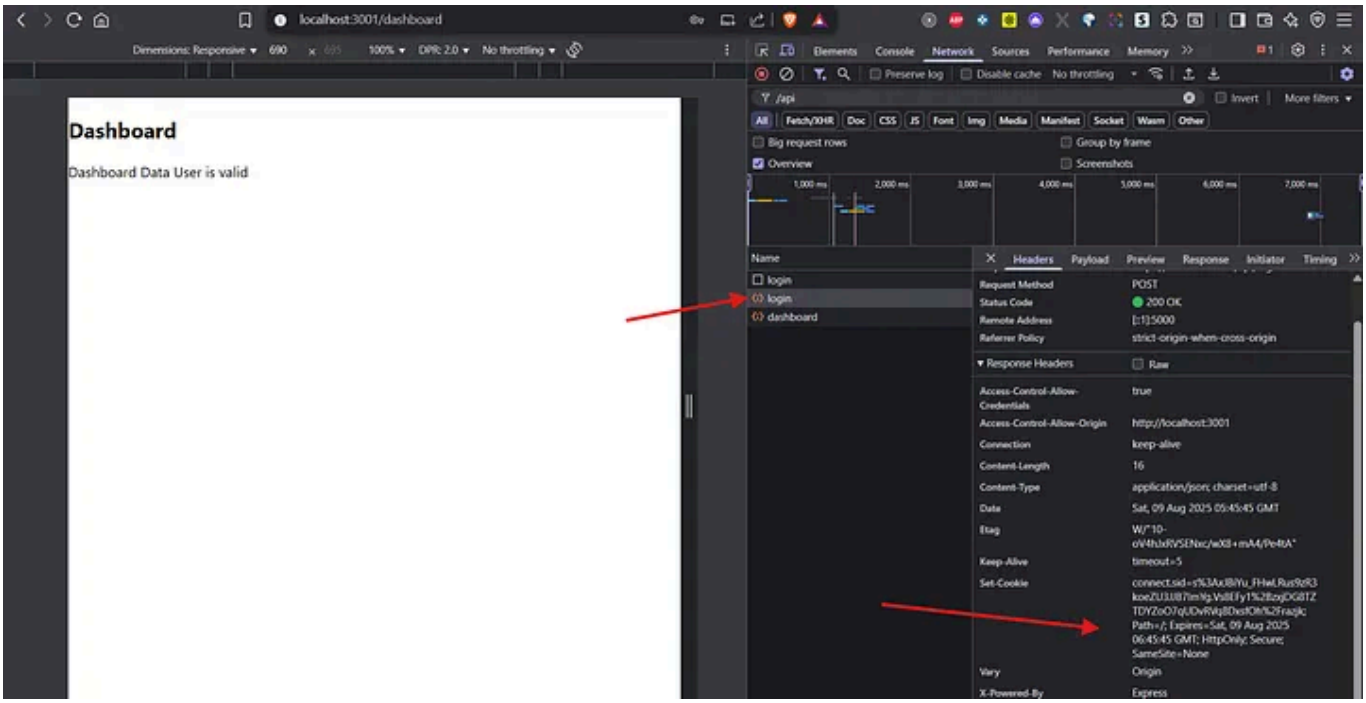
    if (username) {
      res.send({ success: true, message: "Dashboard Data User is valid" });

    } else {
       res.send({ success: false, message: "Please login " });
    }
});

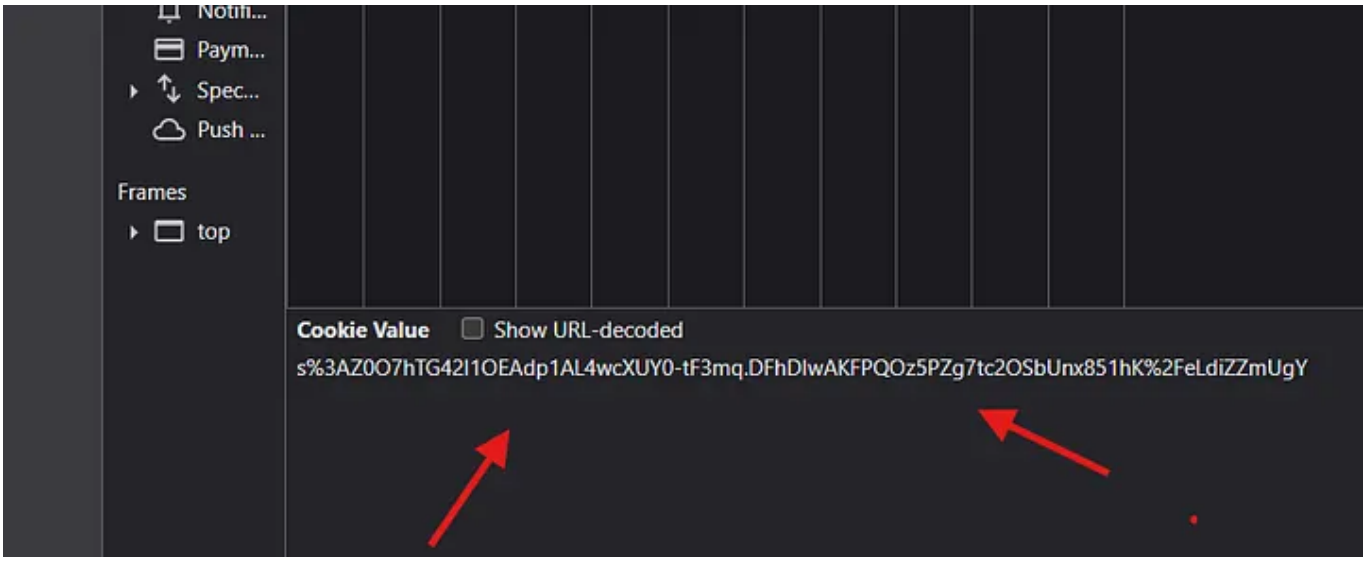
// HTTPS Server Setup
const options = {
    key: fs.readFileSync("server.key"),    // Private key
    cert: fs.readFileSync("server.cert"), // Certificate
};

//to create https server
const PORT = 5000;
https.createServer(options, app).listen(PORT, () => {
    console.log(`HTTPS Server running on https://localhost:${PORT}`);
});
```

Now let’s re login and check the cookie now

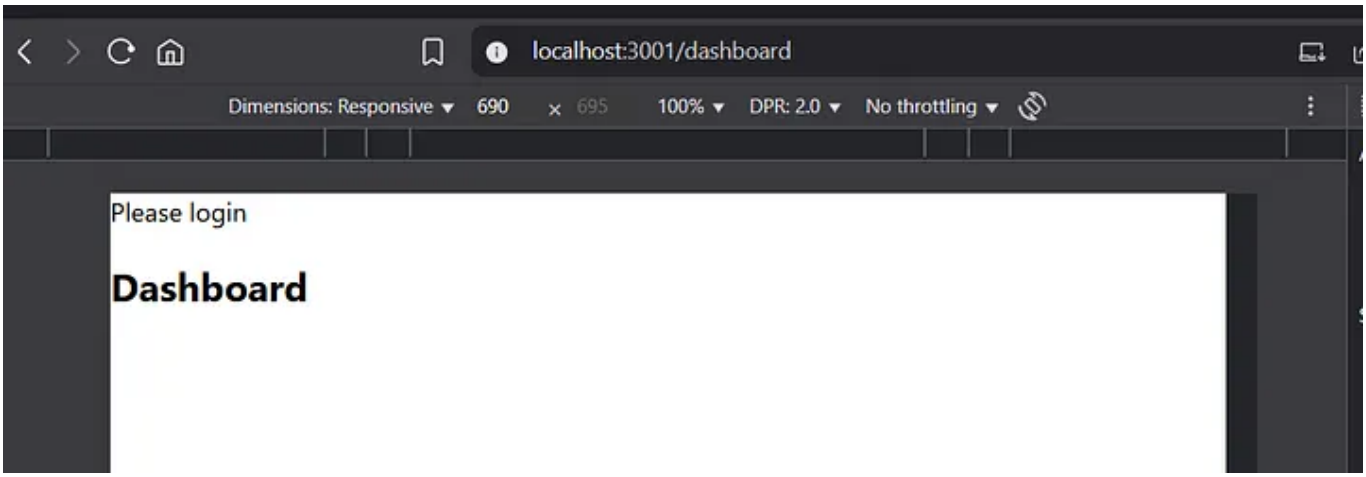


one part is id and other is signature



Now we get cookies as this, not the username





Now we will check the session, instead of the cookie, directly and we get the data which we passed in session object as user

```
app.get("/api/dashboard", (req, res) => {
    const sessionData = req.session || {}
    console.log(sessionData);

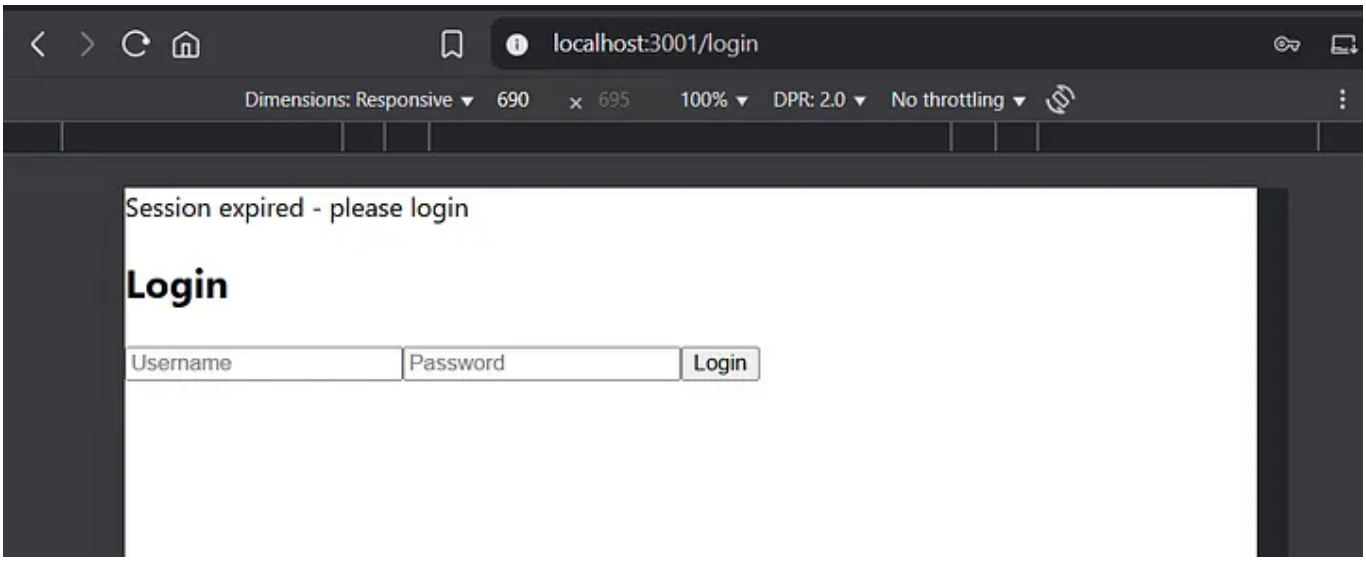
    if (req.session.user.username) {
     res.send({ success: true, message: "Dashboard Data User is valid" });

    } else {
    res.send({ success: false, message: "Please login " });
    }
});
```

```
shubham_gupta@Shubham:~/node-cource$ node express.js
HTTPS Server running on https://localhost:5000
Session {
  cookie: {
    path: '/',
    _expires: 2025-08-09T06:54:10.895Z,
    originalMaxAge: 3600000,
    httpOnly: true,
    secure: true,
    sameSite: 'none'
  },
  user: { username: 'test', id: 123 }
}
```

What happens when I restart the server?

Now, when I restart the server and reload my dashboard page



```
    at cors (/home/shubham_gupta/node-cource/node_modules/cors/lib/index.js:18
8:7)
Session {
  cookie: {
    path: '/',
    _expires: 2025-08-09T06:58:37.716Z,
    originalMaxAge: 3600000,
    httpOnly: true,
    secure: true,
    sameSite: 'none'
  }
}
TypeError: Cannot read properties of undefined (reading 'username')
    at file:///home/shubham_gupta/node-cource/express.js:54:26
    at Layer.handleRequest (/home/shubham_gupta/node-cource/node_modules/route
r/lib/layer.js:152:17)
    at next (/home/shubham_gupta/node-cource/node_modules/router/lib/route.js:
157:13)
    at Route.dispatch (/home/shubham_gupta/node-cource/node_modules/router/lib
```

As you can see, the user data that we added while logging in is gone. And now I am not logged in.

> *Once the server crashes or restarts all sessions are gone , as they are stored in RAM*

So it is better to store your user session data in some persistent storage.

Now, here I will save it inside a file, using a lib session-file-store

```
npm i session-file-store
```

and use it inside our Express.js. Also, bind it to the session where we pass the filestore.

```
import FileStore from "session-file-store"

//create a filestore and bind to session
const fileStore=FileStore(session)

app.use(session(
    {
       // automatically creates folder if not present
        store:new fileStore({path:"./userSessions"}),
        secret: "47f17f0325ef0f15ef889020bcf5de8e",//random key
        resave: false,
        saveUninitialized:false,
        cookie: {
            maxAge: 1000 * 60 * 60,
            httpOnly: true,
            secure: true,       // Must be true for HTTPS
            sameSite: 'none',
        }
    }

));
```

```
import express from "express";
import cookieParser from "cookie-parser";
import https from "https";
import fs from "fs";
import cors from "cors";
import session from "express-session"
import FileStore from "session-file-store"

//create a filestore and bind to session
const fileStore=FileStore(session)

const user = {
    username: "test",
    password: "123"
}
const app = express();

app.use(session(
    {
        store:new fileStore({path:"./userSessions"}),
        secret: "47f17f0325ef0f15ef889020bcf5de8e",//random key
        resave: false,
        saveUninitialized:false,
```

```
        cookie: {
            maxAge: 1000 * 60 * 60,
            httpOnly: true,
            secure: true,       // Must be true for HTTPS
            sameSite: 'none',
        }
    }

));
app.use(express.urlencoded({ extended: true }))
app.use(express.json());
app.use(cookieParser());
app.use(cors({
    origin: 'http://localhost:3001', //add your React dev server
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization']
}));

app.post("/api/login", (req, res) => {
    const { username, password } = req.body
    if (username === user.username && user.password === password) {
        // will tie up unique seesion id here i use username,and id
        // so that it can be used on new api call to indetify the user
        req.session.user = { username,id:123 }
        res.send({ success: true });
    } else {
        res.send({ success: false, message: "Invalid User" });
    }

});

app.get("/api/dashboard", (req, res) => {
    const sessionData = req.session || {}
    console.log(sessionData);

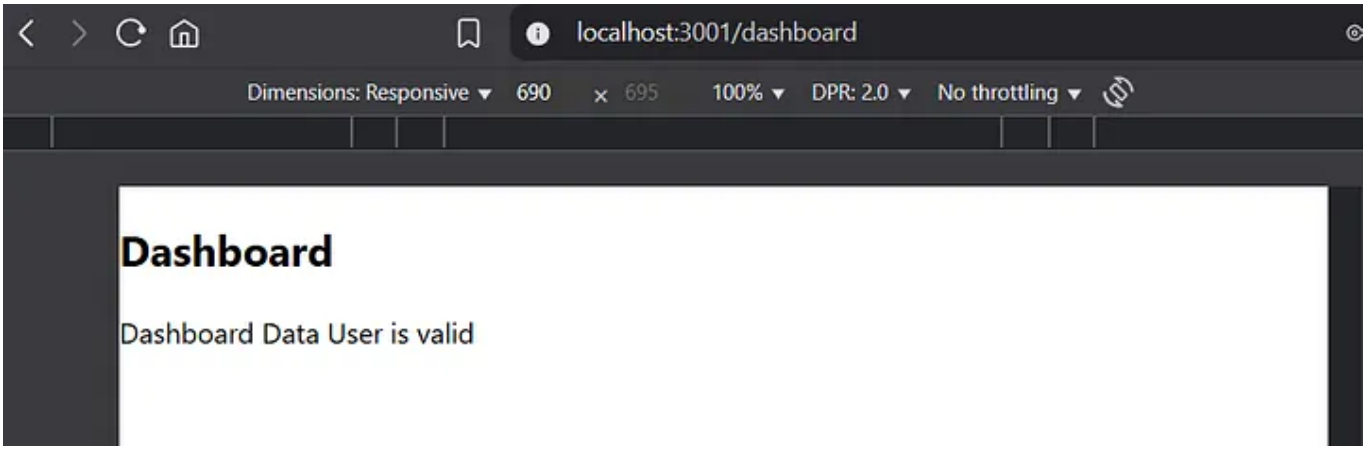
    if (req.session.user.username) {
      res.send({ success: true, message: "Dashboard Data User is valid" });

    } else {
      res.send({ success: false, message: "Please login " });
    }
});

// HTTPS Server Setup
const options = {
    key: fs.readFileSync("server.key"),    // Private key
    cert: fs.readFileSync("server.cert"), // Certificate
};
```

```
//to create https server
const PORT = 5000;
https.createServer(options, app).listen(PORT, () => {
    console.log(`HTTPS Server running on https://localhost:${PORT}`);
});
```

I logged in again



Now let's restart the server and try again, you can see the user persist now even after a restart and without logging in.

```
 file or directory, open 'userSessions/urESQvlxJG2fOYpGPELEtZWmsftYQ_Gl.json'
[session-file-store] will retry, error on last attempt: Error: ENOENT: no such
 file or directory, open 'userSessions/urESQvlxJG2fOYpGPELEtZWmsftYQ_Gl.json'
[session-file-store] will retry, error on last attempt: Error: ENOENT: no such
 file or directory, open 'userSessions/urESQvlxJG2fOYpGPELEtZWmsftYQ_Gl.json'
[session-file-store] will retry, error on last attempt: Error: ENOENT: no such
 file or directory, open 'userSessions/urESQvlxJG2fOYpGPELEtZWmsftYQ_Gl.json'
Session {
  cookie: {
    path: '/',
    _expires: 2025-08-09T07:08:39.446Z,
    originalMaxAge: 3600000,
    httpOnly: true,
    secure: true,
    sameSite: 'none'
  },
  user: { username: 'test', id: 123 },
  __lastAccess: 1754719719447
}
^C
shubham_gupta@Shubham:~/node-cource$ node express.js
HTTPS Server running on https://localhost:5000
Session {
  cookie: {
    path: '/',
    _expires: 2025-08-09T07:08:39.492Z,
    originalMaxAge: 3600000,
    httpOnly: true,
    secure: true,
    sameSite: 'none'
  },
```

Open in app ↗ Sign up Sign in

Medium Search Write



• • •

Options in Express session

resave

- Controls whether the session should be **saved back to the store**, even if **nothing was changed.**
- If true, the session will be saved on every request (bad for performance).
- If false, it will only be saved **if something was modified.**

Better to keep it false.

saveUninitialized

- Controls whether a **new, empty session** should be saved to the store.
- A session is `uninitialized` when it's **new** but **no data has been set** on it yet (req.session is untouched).
- If true, **every visitor** gets a session and a cookie, even if you don't store anything.
- If false, a session is **only created when you set something** in it.

✅ Recommended: saveUninitialized: false

Get Shubham Gupta's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

Remember me for faster sign in

(This avoids setting unnecessary cookies.)

secret

- Used to **sign and encrypt** the session ID cookie.
- This helps prevent tampering by the client.
- It's important to keep this value **secure and private**, especially in production.

🧠 Internally:

- The session ID is signed with this secret so Express can verify it wasn't altered by the client.

cookie

cookie.maxAge	Time (in ms) after which the cookie expires (e.g., 60000 = 1 min)
cookie.secure	If true, cookie only sent over HTTPS (recommended in production)
cookie.httpOnly	If true, JavaScript can't access the cookie (security)
cookie.sameSite	Prevents CSRF attacks (set to 'lax', 'strict', or 'none')

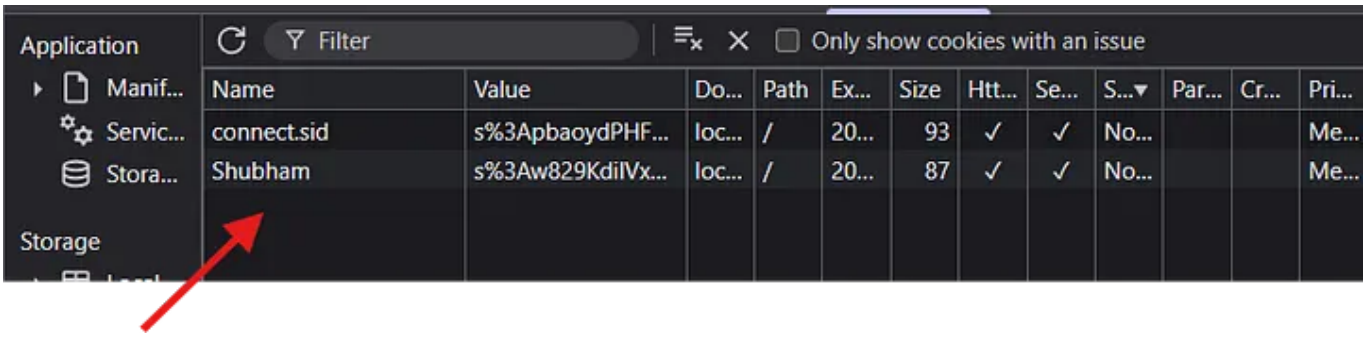
name

Custom name for the session ID cookie (default isconnect.sid)

I added a name property and restarted the server

```
app.use(session(
    {
        store:new fileStore({path:"./userSessions"}),
        secret: "47f17f0325ef0f15ef889020bcf5de8e",//random key
        resave: false,
        saveUninitialized:false,
        cookie: {
            maxAge: 1000 * 60 * 60,
            httpOnly: true,
            secure: true,       // Must be true for HTTPS
            sameSite: 'none',
        },
        name:"Shubham"
    }

));
```



rolling

If true, resets the cookie expiration on every request

Let's say when the user logs in and max age is 1 hr and user wokrs for 50 min then after 10 min it will logout.

store

Let you use Redis, MongoDB, or a file store for persistent session storage

example

```
app.use(session({
  secret: 'secret-key',
  resave: false,
  saveUninitialized: false,
  name: 'myapp.sid',
  cookie: {
    maxAge: 15 * 60 * 1000,  // 15 minutes
    secure: false,           // true in HTTPS
    httpOnly: true,
    sameSite: 'lax'
  }
}));
```

. . .

Best Practices

🔐 1. Use a strong secret

- Required for signing session cookies.
- Must be unpredictable, long, and securely stored (not hardcoded in codebase).
- Example:

```
secret: process.env.SESSION_SECRET ||
                crypto.randomBytes(32).toString('hex')
```

🍪 2. Set secure cookie options

```
cookie: {
  httpOnly: true,     // Prevent JS access to cookie
  secure: true,       // Send only over HTTPS
  sameSite: 'lax',    // Helps prevent CSRF
  maxAge: 15 * 60 * 1000  // Session expiry (e.g. 15 minutes)
}
```

📦 3. Use a production-grade session store

Avoid using the in-memory store (MemoryStore) in production — it:

- Leaks memory
- Can’t handle scale (multiple servers)
- Loses data on restart

Recommended options:

- Redis (connect-redis)
- MongoDB (connect-mongo)
- File store for small apps (session-file-store)

🚫 4. Don’t save uninitialized sessions

- Avoids creating empty sessions for users who never log in.
- Saves resources and avoids setting unnecessary cookies.
- keep saveUninitialized: false

🔁 5. Don’t resave unchanged sessions

- Avoid writing session to store if it wasn’t modified.

- Boosts performance and prevents unnecessary writes.

👥 6. Regenerate the session after login

- Prevents **session fixation attacks.**
- Attackers can't reuse the same session ID to hijack accounts.

```
req.session.regenerate(err => {
  if (!err) {
    req.session.user = user.id;
    res.redirect('/dashboard');
  }
});
```

🔓 7. Log out securely

- Clear cookie to ensure it's removed from browser
- Destroy the session on logout:

```
req.session.destroy(err => {
  res.clearCookie('myapp.sid');
  res.redirect('/');
});
```

🔄 8. Use rolling sessions (optional)

- Reset session expiration on each request: `rolling: true`
- Good for active users
- But can be a security risk for never-expiring sessions

🔍 9. Validate session manually if needed

Example: Prevent access to protected routes

```
function authMiddleware(req, res, next) {
  if (!req.session.user) {
    return res.redirect('/login');
  }
  next();
}
```

🕵️‍♀️ 10. Add session ID fingerprinting (advanced)

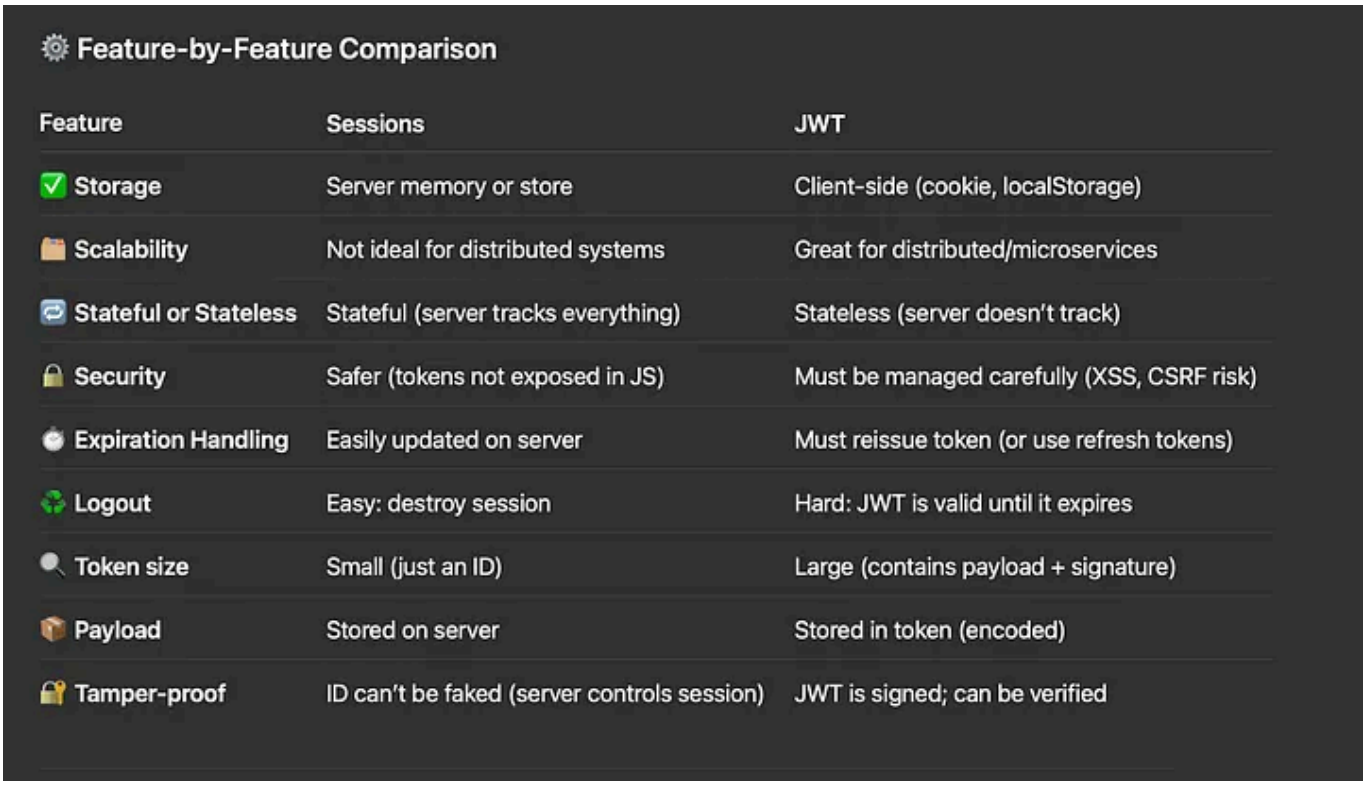
To prevent stolen session cookies from being reused:

```
req.session.fingerprint = req.headers['user-agent'];
  // Store fingerprint data
      req.session.fingerprint = {
        userAgent: req.headers['user-agent'],
        ip: req.ip
      };
```

```
//usage
if (req.session.fingerprint !== req.headers['user-agent']) {
  req.session.destroy();
  res.redirect('/login');
}
```

• • •

vs JWT



⚙️ Feature-by-Feature Comparison

Feature	Sessions	JWT
✅ Storage	Server memory or store	Client-side (cookie, localStorage)
📦 Scalability	Not ideal for distributed systems	Great for distributed/microservices
🔄 Stateful or Stateless	Stateful (server tracks everything)	Stateless (server doesn't track)
🔒 Security	Safer (tokens not exposed in JS)	Must be managed carefully (XSS, CSRF risk)
⏱️ Expiration Handling	Easily updated on server	Must reissue token (or use refresh tokens)
♻️ Logout	Easy: destroy session	Hard: JWT is valid until it expires
🔍 Token size	Small (just an ID)	Large (contains payload + signature)
📦 Payload	Stored on server	Stored in token (encoded)
🔐 Tamper-proof	ID can't be faked (server controls session)	JWT is signed; can be verified

. . .

✅ When to Use Sessions

- Classic monolithic web apps
- You control both the frontend and the backend
- You want server-side invalidation (e.g., logout)
- Easier to secure with HTTP-only cookies

🛠️ Examples:

- E-commerce site
- Admin dashboards
- Express + EJS/Pug or SSR React

✅ When to Use JWT

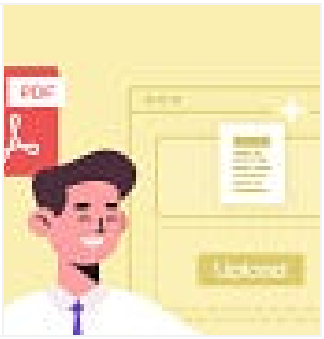
- You need **stateless authentication**
- You have a **mobile app or SPA** that calls APIs
- You're building a **microservice or distributed system**
- You want to authenticate across **multiple domains/services**

. . .

Step-by-Step: File Uploads with Multer in Node.js & Express

(Node Series Part 10–2)

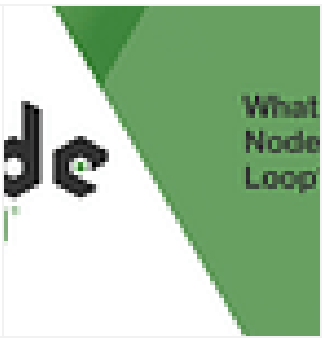
medium.com



Event Loop In Node.js

(Node Series Part 12)

medium.com



. . .

If you'd like to be in the know, subscribe, clap, like and share. Cheers!

Nodejs JavaScript React Javascript Development Node Js Development



Written by Shubham Gupta

195 followers · 13 following

Follow

React Developer

Responses (1)

Write a response

What are your thoughts?

ak editz arun mudhraj
Nov 24, 2025

very informative content

1 Reply

More from Shubham Gupta

Shubham Gupta

Dynamic Imports in React

Dynamic imports in React allow you to dynamically load JavaScript modules at...

Sep 17, 2023 347 4

Shubham Gupta

Features introduced in ES6

ES6 or the ECMAScript 2015 is the 6th and major edition of the ECMAScript language...

Sep 11, 2023 71 1

Shubham Gupta

Step-by-Step: File Uploads with Multer in Node.js & Express

(Node Series Part 10–2)

Aug 3, 2025 8

Shubham Gupta

Mastering Express Router: The Complete Guide to Route Handlin...

(Node Series Part 9)

Jul 28, 2025 2

See all from Shubham Gupta

Recommended from Medium

In Stackademic by Umesh Kumar Yadav

Token, Session, Cookie, JWT, OAuth2—I can't tell the difference!

Recently, I've noticed that some people easily confuse the concepts of Token, Session,...

Dec 10, 2025 214 5

In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

Jan 29 5.7K 49

constCoder

Frontend Interview Challenge: Dynamic Form in React

A few weeks ago, I had a frontend interview for a senior frontend role. It started like any...

Nov 14, 2025 34 1

Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

Jan 19 3.7K 88

In JavaScript in Plain English by Santosh Yadav

I Got Asked These Frontend Architecture Concepts in Almost...

Modern frontend development isn't about frameworks anymore.It's about architecture...

Feb 4 87

Sanjeevani Bhandari

Big Tech Frontend Interview Question: Design a Notification...

A real Big Tech interview question that tests state, events, and UI architecture

Jan 28 89 1

See more recommendations